JAVA SDE-2 INTERVIEW PREPARATION SERIES

VOLUME 18 OF 18 - PART F OF J

Advanced Java F: Design, Backend, Testing, and Security

SOLID APIs, Patterns, Transactions, Services, Build Quality, Reliability, and Secure Boundaries

FOUNDING AUTHOR

Vinay Reddy Kalluri

EDITOR-IN-CHIEF | CHIEF AUDITOR

Connect clean Java design to transactions, service boundaries, testing, build discipline, dependency risk, security, reliability, and operational behavior.

SOLID | LLD | JDBC | TRANSACTIONS | TESTING | SECURITY | RELIABILITY

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-25

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `18E – Advanced Java E – Diagnostics`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Part F covers the engineering judgment that distinguishes SDE-2 candidates from syntax-only candidates: boundaries, invariants, failure policy, observability, and maintainability.

Readiness map

Decision	Evidence
START HERE	A design question has changing policies and integration boundaries; Transaction and retry semantics interact; Tests are brittle or nondeterministic; Security and reliability requirements are implicit.
PREPARE FIRST	Stage 18B language design; Stage 18C libraries; Stage 18D concurrency.
MOVE ON WHEN	Design cohesive APIs and LLD components; Reason about JDBC, transactions, and services; Build testable and diagnosable systems; Apply security and reliability controls at boundaries.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Volume Position

18E - Advanced Java E - Diagnostics	**YOU ARE HERE** **18F - Advanced Java F - Backend**	18G - Advanced Java G - Interview Revision

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Clean Java APIs, SOLID Design, and Low-Level Design Patterns

Learning objectives

By the end of this chapter, you should be able to:

- derive classes and interfaces from domain invariants, use cases, and ownership;
- design small Java APIs with explicit null, mutation, failure, ordering, blocking, and concurrency contracts;
- apply SOLID principles as trade-off questions rather than slogans;
- recognize and implement Strategy, Factory, Adapter, Decorator, Builder, State, Command, and Observer where justified;
- evaluate substitutability, cohesion, coupling, and extension cost; and
- communicate a low-level design from requirements through objects, flows, edge cases, and tests.

WHY IT WORKS | Why this matters at SDE-2

SDE-2 design interviews test whether you can turn ambiguous behavior into a maintainable object model. Production code tests the same skill over years: callers depend on names, defaults, failures, side effects, and timing as much as on types. A clean implementation behind an unclear API still spreads defects.

Good design is not the maximum number of interfaces or patterns. It places volatile decisions behind narrow boundaries, keeps invariants close to state, and makes the common call correct by construction. It also acknowledges operational behavior: an innocent-looking getter that performs network I/O is not clean simply because its name is short.

WHY IT WORKS | First-principles model

Start with behavior and invariants:

TEXT

```
actors and use cases
    -> commands and queries
    -> state and invariants
    -> ownership and lifecycle
    -> failure and concurrency policy
    -> interfaces at real variation boundaries
```

An object is useful when it combines state with operations that preserve that state's valid region. If callers must remember five checks before every mutation, the abstraction has leaked its invariant.

For every public operation, define:

```text
preconditions -> state transition or result -> postconditions
              -> failure mode -> side effects -> ownership
```

Prefer a small number of domain concepts with high cohesion. Coupling is unavoidable; the goal is to couple stable policy to explicit abstractions and keep unstable infrastructure at replaceable edges.

> **Specification boundary:** Java types express some constraints, such as access, generic relationships, and checked exceptions, but they do not express nullness, units, thread safety, blocking, ownership, idempotency, or most value invariants. These are application API contracts and must be encoded through types, validation, tests, and documentation.

Core terminology

- **Invariant:** Condition that must hold for every externally observable valid object state.
- **Precondition:** Requirement a caller must satisfy before an operation.
- **Postcondition:** Guarantee after successful completion.
- **Cohesion:** Degree to which a component's responsibilities belong together.
- **Coupling:** Dependencies between components or knowledge of one another's details.
- **Encapsulation:** Protecting a model by controlling access to representation and transitions.
- **Composition:** Building behavior by containing and delegating to collaborators.
- **Policy:** Business decision or rule.
- **Mechanism:** Technical means used to carry out policy.
- **Dependency inversion:** High-level policy depends on an abstraction rather than a low-level detail.
- **Behavioral subtyping:** A subtype can replace its supertype without violating caller expectations.
- **Pattern:** Named reusable structure for a recurring design pressure, not a mandatory template.
- **Seam:** Boundary where behavior can vary or be tested independently.

Detailed mechanics

Clean API contracts

Names should communicate units and effects: `timeout(Duration)` is stronger than `timeout(int)`, and `loadFromDatabase` is more honest than `get` when I/O occurs. Prefer domain types such as `OrderId`, `Money`, or `PageToken` when primitives would be confused or invalid states are common.

Validate at construction and mutation boundaries. Fail fast with the most specific useful exception when a programming precondition is violated. Domain rejection may deserve a domain result or exception rather than `IllegalArgumentException`. Do not partially mutate an object and then validate.

Return empty collections instead of null for zero results. Use `Optional` for a maybe-one return when absence is normal, not for every field or parameter. State whether collections preserve order, permit duplicates/nulls, are snapshots, or are live views. `List.copyOf` protects membership but does not make mutable elements immutable.

Keep command-query separation as a useful default: queries report state without changing domain state; commands perform transitions and return the minimum needed result. It is acceptable for a command to return an identifier or outcome. Avoid setters that bypass rules. `order.cancel(reason, now)` communicates and enforces more than `setStatus(CANCELED)`.

Checked versus unchecked exceptions is an API decision. Checked exceptions can force handling of recoverable boundary conditions but propagate coupling. Unchecked exceptions suit violated programming contracts and failures callers cannot meaningfully recover from locally. Never discard context; wrap with causal chains and safe domain metadata.

SOLID as five review questions

Single Responsibility Principle: Does this unit have one cohesive reason to change? "One method" is not the rule. A pricing component can contain several methods if all preserve pricing policy. A class mixing pricing, SQL, JSON, retries, and email has unrelated change drivers.

Open/Closed Principle: Can a likely new variant be added behind a deliberate seam without editing a fragile conditional everywhere? Do not create extension points for imagined futures. A sealed hierarchy can intentionally close variants; openness is not universally desirable.

Liskov Substitution Principle: Can every implementation honor the abstraction's preconditions, postconditions, invariants, and failure semantics? A subtype that rejects inputs the base accepts, returns weaker results, or changes a nonblocking call into unbounded blocking is not substitutable even if signatures match.

Interface Segregation Principle: Do clients depend only on capabilities they use? Split read, write, administration, and lifecycle interfaces when those capabilities differ. Avoid one giant service interface, but also avoid dozens of one-method interfaces with no independent variation.

Dependency Inversion Principle: Does high-level policy own the abstraction it needs, while adapters translate database, HTTP, clock, or vendor details? Depending on interfaces is not sufficient if the interface merely mirrors one vendor SDK.

Composition and inheritance

Prefer composition for optional or combinable behavior. Inheritance exposes protected state, constructor ordering, override interactions, and a strong "is-a" behavioral promise. Use it for genuine substitutable families with stable shared semantics. Mark classes or methods final when extension would bypass invariants.

Records suit transparent immutable data aggregates, not automatically rich entities. Their components are shallowly final; contained collections and objects may mutate. Sealed types model a known set of variants and enable exhaustive reasoning. Interfaces model capabilities across unrelated implementations.

> **HotSpot note:** Whether a composed call, interface call, or small value object allocates at runtime depends on profiling, inlining, and escape analysis. Do not distort an API solely to predict HotSpot optimization. Measure a confirmed hot path on the target JDK.

Patterns and their design pressure

Pattern	Pressure it addresses	Common misuse
Strategy	Select one interchangeable policy	interface for behavior that never varies
Factory	Centralize valid creation and implementation selection	hiding arbitrary service location
Adapter	Translate an external or legacy contract	leaking vendor types through the adapter
Decorator	Add composable behavior around one contract	changing core semantics unexpectedly
Builder	Construct many optional or staged parameters	mutable builder reused across threads
State	Behavior changes with explicit lifecycle state	replacing a small clear switch with many classes
Command	Represent an action for queueing, logging, undo, or dispatch	turning every method call into an object
Observer	Notify multiple dependents	unclear lifetime, ordering, and failure isolation
Template Method	Share an algorithm skeleton through inheritance	fragile override hooks

Patterns can combine. A factory creates a strategy; decorators add metrics or retries; an adapter implements the strategy using a vendor. Each layer must preserve the underlying contract. Retry, for example, is safe only for idempotent operations or when an idempotency mechanism exists.

Low-level design interview method

Clarify scale, actors, operations, consistency, concurrency, and out-of-scope requirements. Identify nouns but validate them through behavior; not every noun needs a class. State invariants and lifecycle transitions. Sketch public APIs and one critical sequence. Then examine extension points, storage boundaries, failure, thread safety, and test seams.

Discuss trade-offs explicitly. A map-based in-memory design may be correct for one-process scope but not durable or distributed. Do not smuggle distributed guarantees into a class diagram. Keep the initial model runnable and explain how boundaries evolve.

TRACE IT | Worked Java example

This pricing design keeps money invariants in a value object and discount variation behind a Strategy:

JAVA (continued 1/2)

```java
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.List;

record Money(BigDecimal amount, String currency) {
    public Money {
        if (amount == null || currency == null || currency.isBlank()) {
            throw new IllegalArgumentException("money fields are required");
        }
        amount = amount.setScale(2, RoundingMode.UNNECESSARY);
        if (amount.signum() < 0) {
            throw new IllegalArgumentException("amount must be non-negative");
        }
    }

    Money add(Money other) {
        requireSameCurrency(other);
        return new Money(amount.add(other.amount), currency);
    }

    Money subtract(Money other) {
        requireSameCurrency(other);
        return new Money(amount.subtract(other.amount), currency);
    }

    Money multiply(int quantity) {
        if (quantity < 0) throw new IllegalArgumentException("negative quantity");
```

JAVA (continued 2/2)

```
        return new Money(amount.multiply(BigDecimal.valueOf(quantity)), currency);
    }

    private void requireSameCurrency(Money other) {
        if (!currency.equals(other.currency)) {
            throw new IllegalArgumentException("currency mismatch");
        }
    }
}

record LineItem(String sku, int quantity, Money unitPrice) {
    public LineItem {
        if (sku == null || sku.isBlank() || quantity <= 0 || unitPrice == null) {
            throw new IllegalArgumentException("invalid line item");
        }
    }
}

record Order(List<LineItem> items, String customerTier) {
    public Order {
        items = List.copyOf(items);
        if (items.isEmpty() || customerTier == null) {
            throw new IllegalArgumentException("invalid order");
        }
    }
}
```

The policy contract and pricing service build on those value objects:

JAVA (continued 1/2)

```
interface DiscountPolicy {
    Money discountFor(Order order, Money subtotal);
}

final class TierDiscountPolicy implements DiscountPolicy {
    @Override
    public Money discountFor(Order order, Money subtotal) {
        BigDecimal rate = switch (order.customerTier()) {
            case "GOLD" -> new BigDecimal("0.10");
            case "SILVER" -> new BigDecimal("0.05");
            default -> BigDecimal.ZERO;
        };
        BigDecimal amount = subtotal.amount().multiply(rate)
                .setScale(2, RoundingMode.HALF_EVEN);
        return new Money(amount, subtotal.currency());
    }
}

record Quote(Money subtotal, Money discount, Money total) {}
```

JAVA (continued 2/2)

```java
final class PricingService {
    private final DiscountPolicy discountPolicy;

    PricingService(DiscountPolicy discountPolicy) {
        this.discountPolicy = java.util.Objects.requireNonNull(discountPolicy);
    }

    Quote quote(Order order) {
        String currency = order.items().get(0).unitPrice().currency();
        Money subtotal = new Money(BigDecimal.ZERO.setScale(2), currency);
        for (LineItem item : order.items()) {
            subtotal = subtotal.add(item.unitPrice().multiply(item.quantity()));
        }
        Money discount = discountPolicy.discountFor(order, subtotal);
        if (!discount.currency().equals(currency)
                || discount.amount().compareTo(subtotal.amount()) > 0) {
            throw new IllegalStateException("discount policy violated its contract");
        }
        return new Quote(subtotal, discount, subtotal.subtract(discount));
    }
}
```

The service owns orchestration; the policy owns discount variation; `Money` owns currency and nonnegative-value rules. A factory could select a policy from configuration, and a decorator could measure policy latency without changing `PricingService`.

TRACE IT | Execution or memory walkthrough

For two USD items priced `10.00 x 2` and `5.00 x 1`, `PricingService` starts with USD zero, produces subtotals `20.00` and then `25.00`, and asks the policy for a discount. A GOLD order yields `2.50`, so the quote total is `22.50`.

Construction copies the item list, preventing later membership changes through the caller's list. `LineItem` and `Money` are immutable records, so this particular object graph is stable. If a component were a mutable object, `List.copyOf` alone would not deep-copy it.

The service checks the strategy's postcondition: same currency and discount no greater than subtotal. That defensive boundary makes a broken implementation fail near the cause rather than creating negative totals downstream. It does not need to know how the strategy chose the discount.

Complexity and performance

For `n` line items, quote computation is `O(n)` time and `O(1)` additional domain storage beyond immutable result objects. `BigDecimal` arithmetic cost depends on precision and magnitude, so the simple `O(n)` model treats money operation size as bounded.

Abstraction cost should be considered at the design level first:

Choice	Benefit	Cost to evaluate
immutable values	safe sharing, simple invariants	allocation and copying
interface seam	substitution and testing	more concepts and contract surface
defensive copy	ownership isolation	`O(n)` time and references
decorator	composable cross-cutting behavior	call depth, ordering interactions
builder	readable optional construction	extra mutable lifecycle

Avoid making every field a wrapper or every class an interface in the name of cleanliness. Complexity of understanding is a performance constraint on the engineering organization. Measure runtime concerns only after profiling identifies a hot path.

WATCH OUT | Edge cases and common mistakes

- Starting from patterns instead of use cases and invariants.
- Creating interfaces for every class with no independent implementation or test seam.
- Using setters that permit invalid intermediate states.
- Hiding blocking I/O or mutation behind query-like names.
- Returning internal mutable collections or mutable builder state.
- Treating record components as deeply immutable.
- Applying LSP only to method signatures and ignoring failure, timing, and side effects.
- Building a base class with protected fields and many override hooks.
- Adding retry decorators to non-idempotent operations.
- Letting an adapter expose vendor DTOs, exceptions, or configuration throughout the domain.
- Using a service locator or static singleton and calling it dependency inversion.
- Splitting cohesive behavior across so many classes that one change requires navigating a graph.
- Using `double` for money or omitting currency and rounding contracts.
- Optimizing dispatch based on guessed JVM behavior.

Production engineering notes

Treat public APIs as versioned products. Remove ambiguity about nulls, blocking, timeout, idempotency, thread safety, collection ownership, ordering, and exceptions. Favor additive evolution and migration adapters over silently changing semantics.

Keep domain policy free of framework annotations and vendor DTOs where practical. Translate at controllers, messaging adapters, and repositories. This makes business behavior testable and limits dependency upgrades to boundary code. Do not create duplicate models mechanically when one stable representation genuinely serves both layers.

Lifecycle is part of design. Observers need registration handles; executors and clients need closure; caches need bounds; decorators need ordering; factories need validated configuration. Expose health and metrics through separate operational interfaces rather than mixing administrative mutation with core use cases.

Run architecture checks for forbidden dependency directions, but use reviews to evaluate semantics. Static package rules cannot prove substitutability or a useful abstraction. Record significant design decisions, alternatives, and reversal cost.

TRY IT | Interview questions and model answers

What makes an API clean?

It has cohesive operations, explicit invariants and ownership, unsurprising names, minimal invalid states, and documented failure, blocking, concurrency, null, and ordering behavior. It is easy to use correctly and hard to misuse.

Explain dependency inversion with an example.

Pricing policy depends on a small exchange-rate abstraction owned by the domain, not a vendor HTTP client. An adapter implements that abstraction using the vendor. Policy tests use a deterministic implementation, and vendor changes stay at the edge.

How do you decide between composition and inheritance?

Use inheritance only for a true behavioral subtype with stable shared semantics. Use composition for optional, replaceable, or combinable behavior because it exposes fewer override interactions and preserves encapsulation.

Is SOLID always good?

The principles are review heuristics with trade-offs. Premature interfaces and tiny classes can increase cognitive load. Apply them at observed change boundaries and preserve cohesion.

How do Strategy and Decorator differ?

Strategy selects the core policy implementation. Decorator wraps the same contract to add behavior such as metrics, caching, or authorization while delegating. A decorator must preserve the wrapped contract.

How would you approach a low-level design interview?

Clarify use cases and scale, state invariants, model lifecycle and ownership, sketch the public API and critical sequence, then cover extension, concurrency, failure, persistence boundaries, complexity, and tests.

TRY IT | Exercises

1. Replace a `setStatus` order API with valid transition methods and specify preconditions and failures.
2. Design a retry decorator contract that is safe only for idempotent commands. Show how callers communicate idempotency.
3. Refactor a report class that queries SQL, calculates totals, formats JSON, and emails output into cohesive boundaries.
4. Add a promotion-composition policy to the worked example. Decide whether discounts stack, cap, or select the best.
5. Model a vending machine with either a State pattern or an enum switch. Compare extension and readability.
6. Review an interface whose implementations disagree on null, timeout, and exception behavior. Write a substitutable contract.

RECAP | Chapter summary

Clean Java design starts with use cases, invariants, ownership, and observable contracts. SOLID principles expose change, substitutability, capability, and dependency questions; they are not a class-count target. Patterns name structures that address real variation or lifecycle pressure, and composition is usually the safest default. A strong low-level design remains runnable, states trade-offs, isolates infrastructure, and makes invalid state and accidental side effects difficult.

TRY IT | Revision checklist

- **[ ]** I derive types and methods from behavior and invariants rather than nouns alone.
- **[ ]** I document null, mutation, order, failure, blocking, idempotency, lifecycle, and thread safety.
- **[ ]** I can apply all five SOLID principles as concrete review questions.
- **[ ]** I distinguish behavioral subtyping from signature compatibility.
- **[ ]** I prefer composition unless a genuine stable is-a relationship exists.
- **[ ]** I can select and critique common low-level design patterns.
- **[ ]** I keep policy independent from vendor and framework details at deliberate seams.
- **[ ]** I can present requirements, APIs, sequence, edge cases, complexity, and tests in a design interview.

SDE-2 CORE CHAPTER

Chapter 2 - Backend Java Boundaries: JDBC, Transactions, Serialization, and Services

Learning objectives

By the end of this chapter, you should be able to:

- use JDBC resources and prepared statements with explicit ownership and transaction control;
- choose transaction boundaries from business invariants rather than repository method boundaries;
- explain isolation anomalies and why database implementations must be verified;
- avoid dual-write inconsistency with transactional outbox and idempotent processing patterns;
- design versioned, validated serialization DTOs independent from persistence entities; and
- specify timeouts, retries, idempotency, backpressure, and observability at service boundaries.

WHY IT WORKS | Why this matters at SDE-2

Backend correctness lives between components. A Java method can be locally correct while its transaction commits half an invariant, its JSON change breaks an older consumer, or its retry charges a customer twice. JDBC, messaging, HTTP, and serialization are not plumbing details; they define consistency and failure semantics.

At SDE-2, you are expected to place a transaction around a use case, explain what it cannot include atomically, and make remote effects retry-safe. You should know that closing a pooled connection usually returns it rather than closing a socket, that `PreparedStatement` protects values rather than arbitrary SQL identifiers, and that a timeout does not prove the remote operation did not happen.

WHY IT WORKS | First-principles model

A boundary converts one model and failure domain into another:

```text
HTTP bytes -> validated request DTO -> application command
           -> domain transition -> SQL transaction
           -> outbox record -> broker message -> remote consumer
```

Each arrow needs a contract: encoding, validation, identity, timeout, atomicity, retry, ordering, ownership, and version compatibility.

A local database transaction groups statements under ACID goals:

- atomicity: all transaction effects commit or none do;
- consistency: constraints and application rules move valid state to valid state;
- isolation: concurrent transactions observe behavior defined by an isolation level;
- durability: committed changes survive according to database guarantees and configuration.

The database transaction does not automatically include an HTTP call, email, cache, or message broker. Crossing failure domains requires protocols and recoverable state, not a larger Java `try` block.

> **Specification boundary:** JDBC standardizes interfaces and broad transaction operations. SQL syntax, type mappings, generated keys, isolation behavior, lock semantics, timeout enforcement, batch behavior, and connection-pool reset are database, driver, and pool specific. Test the exact versions in use.

Core terminology

- **DataSource:** Factory for database connections; often backed by a pool.
- **Connection:** JDBC session and transaction context.
- **Prepared statement:** Precompiled/parameterized statement separating SQL structure from values.
- **Transaction boundary:** Scope whose database effects commit or roll back together.
- **Isolation anomaly:** Concurrent outcome such as dirty read, nonrepeatable read, phantom, lost update, or write skew.
- **Optimistic concurrency:** Detect conflict using a version or compare-and-set condition.
- **Pessimistic locking:** Acquire database locks before a conflicting transition.
- **Idempotency:** Repeating an operation with the same identity has the intended single logical effect.
- **Outbox:** Database table storing events in the same local transaction as domain changes.
- **Inbox/deduplication:** Consumer record of processed message identities.
- **DTO:** Boundary-specific data transfer object.
- **Schema evolution:** Changing a wire or storage representation while preserving compatibility policy.
- **Retry budget:** Bound on attempts, elapsed time, and load amplification.

Detailed mechanics

JDBC resource ownership

Acquire a connection as late as practical and release it promptly. `Connection`, `Statement`, and `ResultSet` are `AutoCloseable`; nested try-with-resources closes them in reverse order. Closing a connection from a pool normally returns it to the pool, but that behavior belongs to the configured `DataSource`.

Do not hold a database connection while performing slow remote I/O. It extends lock time and consumes scarce pool capacity. A pool is a concurrency bound, not a throughput generator. Monitor active, idle, wait time, timeouts, and leaked borrows.

Use prepared statements for values:

JAVA

```java
try (var statement = connection.prepareStatement(
        "select id, status from orders where customer_id = ?")) {
    statement.setString(1, customerId);
    try (var rows = statement.executeQuery()) {
        while (rows.next()) {
            // Map explicitly by stable column labels.
        }
    }
}
```

Placeholders do not represent table names, column names, sort direction, or arbitrary SQL fragments. Dynamic identifiers require a fixed allow-list mapped to known SQL. Never concatenate untrusted input.

Set query, lock, transaction, and network timeouts according to driver capabilities and an end-to-end deadline. `Statement.setQueryTimeout` units and enforcement have driver limits. A canceled query may continue server-side briefly; monitor the database as well as the client.

Transaction boundaries and failure handling

Disable auto-commit before a multi-statement transaction, commit once after all invariant checks and writes, and roll back on failure. If rollback throws, preserve it as a suppressed exception rather than losing the primary cause. Never return a connection with an open transaction or altered session state; use a pool that reliably resets documented state and keep application behavior disciplined.

The boundary belongs at the application use case. Two repository methods called separately with independent transactions cannot enforce a cross-table invariant. Conversely, a transaction spanning user think time or a remote request holds resources and locks too long.

Database constraints are the final concurrent guard. Use primary keys, unique constraints, foreign keys, checks, and atomic update predicates. A Java "check then insert" can race. Map constraint violations to domain outcomes carefully using driver/database error information rather than brittle message parsing.

Isolation and concurrency control

Isolation levels are commonly described by prohibited phenomena, but real databases use locks or multiversion concurrency control with vendor-specific behavior. "Repeatable read" does not mean identical semantics everywhere. Serializable execution can abort a transaction and require retry.

For an update such as reserving inventory, useful approaches include:

- atomic conditional SQL: update where available quantity is sufficient, then inspect update count;
- optimistic versioning: update where `version = expected`, increment version, reject count zero;
- pessimistic lock: lock the row, inspect, then update within one short transaction;
- serializable transaction: let the database detect conflicting executions and retry safe failures.

Choose from contention, invariant shape, latency, and database behavior. Retrying a transaction reruns its code, so no non-idempotent external side effect should occur inside it.

> **Vendor boundary:** Lock clauses, deadlock detection, snapshot rules, serialization failures, timestamp precision, UUID/JSON types, and DDL transaction behavior differ. Treat database documentation and integration tests as part of the contract.

The dual-write problem

Suppose the service commits an order and then publishes `OrderCreated`. A crash between those actions loses the event. Publishing first can expose an event for an order that later rolls back. Ordinary local transactions cannot atomically cover both systems.

The transactional outbox writes the order and an outbox row in one database transaction. A separate relay reads committed rows, publishes messages, and marks progress. Publication may occur more than once around failures, so consumers use stable event IDs and idempotent inbox or natural-key effects. This provides recoverable at-least-once processing, not magical exactly-once behavior across arbitrary side effects.

An outbox needs cleanup, lag metrics, partition/order policy, claim/lease semantics, and poison-message handling. Change-data-capture can relay rows but remains an operational component with its own offsets and failure modes.

Serialization boundaries

Do not serialize persistence entities or rich domain objects directly by default. Lazy-loading proxies, bidirectional graphs, internal fields, and refactors make unstable contracts. Define request/response/event DTOs and map explicitly at the boundary.

Specify charset, media type, field names, required/optional/default behavior, numeric range and precision, timestamps and timezone, enum evolution, unknown fields, null versus absent, collection limits, and maximum nesting/bytes. Validate syntactic shape at decoding and domain rules in the application/domain layer.

Backward compatibility usually means new producers avoid removing or changing fields older consumers require, while consumers tolerate documented additions. Adding an enum value can break exhaustive older consumers. Version semantics, not merely a `/v2` path, must be planned.

Native Java serialization is unsuitable for untrusted service boundaries. Prefer an explicit schema and safe parser configuration. No format is automatically safe: JSON and binary parsers still need size, depth, duplicate-field, polymorphism, and resource limits.

Remote service boundaries

Every call needs an end-to-end deadline propagated through connection acquisition, connect, request, and response phases. Layered timeouts should leave callers time to recover. Cancellation and interruption should release resources.

Retry only failures that are transient and operations that are idempotent or protected by an idempotency key. Use exponential backoff with jitter and a strict attempt/deadline budget. Retries amplify load during an outage; combine them with admission control, circuit breaking, concurrency limits, and bounded queues.

A timeout is an unknown outcome. The server may have committed after the client stopped waiting. Reconcile using an idempotency key and status lookup rather than issuing a semantically new request. Record request IDs and trace context, but do not place secrets or unbounded tenant IDs in metric labels.

Framework transaction caveats

Framework annotations often implement transactions through proxies or interception. Method visibility, self-invocation, exception type, propagation, asynchronous boundaries, and reactive execution can affect whether a transaction exists. These are framework/version rules, not Java or JDBC guarantees. Verify with integration tests that inspect committed outcomes, not only mocked repository calls.

TRACE IT | Worked Java example

This service writes an order and outbox record atomically using standard JDBC:

JAVA (continued 1/2)

```
import java.math.BigDecimal;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.sql.Timestamp;
import java.time.Clock;
import java.time.Instant;
import java.util.UUID;
import javax.sql.DataSource;

record PlaceOrder(String customerId, BigDecimal total, String currency) {}

public final class OrderApplicationService {
    private final DataSource dataSource;
    private final Clock clock;

    public OrderApplicationService(DataSource dataSource, Clock clock) {
        this.dataSource = java.util.Objects.requireNonNull(dataSource);
        this.clock = java.util.Objects.requireNonNull(clock);
    }

    public String place(PlaceOrder command) throws SQLException {
```

JAVA (continued 2/2)

```
        validate(command);
        String orderId = UUID.randomUUID().toString();
        String eventId = UUID.randomUUID().toString();
        Instant now = clock.instant();

        try (Connection connection = dataSource.getConnection()) {
            connection.setAutoCommit(false);
            try {
                insertOrder(connection, orderId, command, now);
                insertOutbox(connection, eventId, orderId, now);
                connection.commit();
                return orderId;
            } catch (SQLException | RuntimeException failure) {
                try {
                    connection.rollback();
                } catch (SQLException rollbackFailure) {
                    failure.addSuppressed(rollbackFailure);
                }
                throw failure;
            }
        }
    }
```

The insert helpers and validation complete the same `OrderApplicationService` class:

JAVA (continued 1/2)

```
    private static void insertOrder(Connection connection, String orderId,
            PlaceOrder command, Instant now) throws SQLException {
        String sql = "insert into orders "
                + "(id, customer_id, total, currency, created_at) "
                + "values (?, ?, ?, ?, ?)";
        try (PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setString(1, orderId);
            statement.setString(2, command.customerId());
            statement.setBigDecimal(3, command.total());
            statement.setString(4, command.currency());
            statement.setTimestamp(5, Timestamp.from(now));
            if (statement.executeUpdate() != 1) {
                throw new SQLException("order insert affected unexpected row count");
            }
        }
    }

    private static void insertOutbox(Connection connection, String eventId,
            String orderId, Instant now) throws SQLException {
        String sql = "insert into outbox "
                + "(event_id, event_type, aggregate_id, payload, created_at) "
```

JAVA (continued 2/2)

```
                + "values (?, ?, ?, ?, ?)";
        try (PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setString(1, eventId);
            statement.setString(2, "OrderCreated");
            statement.setString(3, orderId);
            statement.setString(4, "order-created:" + orderId);
            statement.setTimestamp(5, Timestamp.from(now));
            if (statement.executeUpdate() != 1) {
                throw new SQLException("outbox insert affected unexpected row count");
            }
        }
    }

    private static void validate(PlaceOrder command) {
        if (command == null || command.customerId() == null
                || command.total() == null || command.total().signum() < 0
                || command.currency() == null) {
            throw new IllegalArgumentException("invalid order command");
        }
    }
}
```

The textual payload is deliberately minimal; a real event uses a versioned serializer and schema. SQL types and timestamp mappings must be integration-tested with the selected driver.

TRACE IT | Execution or memory walkthrough

1. Validation runs before borrowing a scarce connection.
2. IDs and time are created once, making retry identity a design decision. As written, a caller retry invokes `place` again and creates a new order; a public idempotency key should be supplied for retry-safe API behavior.
3. Auto-commit is disabled. The order insert and outbox insert share one connection and transaction.
4. If both affect one row, commit makes both visible under database semantics.
5. If the second insert fails, rollback removes the first insert. A rollback failure is preserved as suppressed evidence.
6. A commit or close exception can leave the caller uncertain whether commit succeeded. With a pool, close should return and reset the connection according to pool configuration; public retry safety still requires a stable idempotency key.
7. Later, a relay publishes the event. A crash after broker acceptance but before marking progress can cause duplicate delivery; consumers deduplicate by `eventId`.

The method retains only command and small JDBC objects. The database, not the Java heap, owns durable state. Holding the transaction open while calling the broker would increase lock and pool occupancy without creating cross-system atomicity.

Complexity and performance

Application-side computation is `O(1)` for fixed-size fields, but database cost depends on indexes, constraints, logging, contention, and network latency. An indexed primary-key insert is often described as `O(log n)` for a B-tree, yet storage engines batch, cache, split pages, and write logs; measure the actual database.

Important capacity relationships are:

```text
concurrent database work <= connection-pool capacity
queueing wait + query time + commit time <= request deadline
retry traffic <= remaining downstream capacity
outbox production rate <= sustainable relay rate over time
```

Batching can reduce round trips but changes failure attribution and memory. Large transactions retain locks/versions/log records longer and make retries more expensive. N+1 queries turn one logical operation into `O(n)` round trips; fetch or batch deliberately while avoiding explosive joins.

WATCH OUT | Edge cases and common mistakes

- Building SQL by concatenating values or untrusted sort identifiers.
- Forgetting to close a result set, statement, or connection on an exception path.
- Holding a connection across HTTP calls or user think time.
- Placing each repository call in an independent transaction when the invariant spans calls.
- Assuming application checks replace unique or check constraints.
- Retrying all SQL exceptions, including permanent constraints and unknown side effects.
- Assuming timeout means the database or remote service rolled back.
- Publishing an event after commit with no durable recovery record.
- Claiming exactly-once effects because a broker has an exactly-once feature.
- Serializing ORM entities, lazy proxies, or internal exception objects directly.
- Changing enum, timestamp, numeric, null, or unknown-field semantics without compatibility testing.
- Returning database error details or SQL in public responses.
- Using an annotation without verifying proxy, propagation, and rollback behavior.
- Making queues and retry attempts unbounded during a downstream outage.

Production engineering notes

Size connection pools from database capacity and measured service time, not application thread count. Set acquisition deadlines and expose saturation. Validate connections and pool reset behavior. Keep transactions short, but never split an invariant merely to lower a timing metric.

Use migration tooling with ordered, reviewed, backward-compatible changes. Deploy expand/migrate/contract sequences when old and new application versions overlap. Index creation and DDL locking are database-specific operational events, not harmless startup steps.

Version events and APIs, retain contract fixtures, and run consumer compatibility tests. Redact logs, cap body size, reject invalid content types/charsets, and avoid generic polymorphic deserialization from untrusted input.

For every remote dependency, publish deadline, retry, idempotency, circuit, concurrency, and fallback policy. Measure attempts separately from logical requests, and measure outbox age, retries, poison records, deduplication conflicts, and end-to-end delivery latency.

TRY IT | Interview questions and model answers

Where should a transaction boundary be?

Around the shortest database scope that enforces one business invariant or use case. It should include all required local reads and writes, but not remote I/O that the database cannot commit atomically.

How do prepared statements prevent SQL injection?

They separate SQL structure from bound values so values are not parsed as SQL syntax. They do not parameterize identifiers or arbitrary fragments; dynamic structure must come from an allow-list.

What is the dual-write problem?

Updating a database and publishing to another system cannot be made atomic by ordinary local transaction code. A crash between operations creates inconsistency. A transactional outbox stores publish intent with the domain update and supports recoverable at-least-once relay.

How do you handle duplicate messages?

Give each logical event a stable ID and make the consumer effect idempotent using an inbox/unique constraint or natural idempotent update in the same transaction as its business effect.

What does a timeout tell you?

Only that the caller stopped waiting within its policy. The remote operation may not have started, may still run, or may have committed. Use idempotency and status reconciliation.

How do you choose an isolation level?

Start from the invariant and concurrent anomalies that must be prevented, then choose an atomic statement, optimistic version, lock, or isolation level supported by the target database. Test contention and retry behavior.

TRY IT | Exercises

1. Implement an optimistic update using `where id = ? and version = ?`; interpret update count zero.
2. Add a caller-provided idempotency key to the worked example with a unique database constraint and replayed-result behavior.
3. Design an outbox relay claim algorithm and state what happens after publish succeeds but progress update fails.
4. Specify a versioned `OrderCreated` schema, including time, money, enum, unknown field, and size behavior.
5. Analyze a request whose 500 ms deadline includes a 200 ms pool wait and two retries. Construct a valid remaining-time budget.
6. Write an integration test that proves a framework transaction rolls back both rows when the outbox insert fails.

RECAP | Chapter summary

Backend boundaries convert models and failure domains. JDBC code must own resources, parameterize values, keep transactions short, and rely on database constraints for concurrent truth. Isolation is an invariant decision with vendor-specific behavior. Database and broker effects require a recoverable outbox plus idempotent consumers, not a claim of universal exactly-once delivery. Serialization and remote APIs need explicit version, validation, timeout, retry, and unknown-outcome contracts.

TRY IT | Revision checklist

- [ ] I close JDBC resources and avoid holding connections across remote I/O.
- [ ] I use prepared values and allow-list dynamic SQL structure.
- [ ] I place transactions around use-case invariants and preserve rollback failures.
- [ ] I can compare atomic updates, optimistic versions, locks, and isolation levels.
- [ ] I understand dual-write failure, outbox relay, and consumer idempotency.
- [ ] I define DTO schema, limits, timestamps, money, enum, null, and compatibility behavior.
- [ ] I treat timeout as an unknown outcome and retries as bounded load amplification.
- [ ] I verify database, driver, pool, and framework behavior with integration tests.

SDE-2 CORE CHAPTER

Chapter 3 - Testing, Build Tools, Static Analysis, and Dependency Management

Learning objectives

By the end of this chapter, you should be able to:

- choose unit, integration, contract, end-to-end, property, and performance tests from risk;
- write deterministic behavioral tests with clear fixtures, boundaries, and oracles;
- use fakes, stubs, mocks, clocks, and seeded randomness deliberately;
- configure Java toolchains and release targets for repeatable builds;
- integrate compiler checks, static analysis, formatting, coverage, and mutation evidence without metric gaming; and
- govern direct and transitive dependencies for compatibility, security, licensing, and reproducibility.

WHY IT WORKS | Why this matters at SDE-2

SDE-2 engineers own confidence, not only code. A test suite must catch meaningful regressions quickly enough to run, a build must produce the same intended artifact in CI and release, and dependencies must be understood as executable supply-chain input.

Many teams have thousands of tests and still fear deployment because tests assert implementation details, integration behavior is mocked away, and flaky timing tests are ignored. Other teams create slow end-to-end suites for behavior that a pure unit test would prove better. The goal is a layered evidence system whose cost matches risk.

WHY IT WORKS | First-principles model

A test is an experiment:

```text
given controlled state and inputs
when one behavior occurs
then observable results and side effects match an oracle
```

A useful suite varies the scope of that experiment:

```text
small pure tests        -> fast logic feedback
component/integration   -> database, serialization, framework wiring
contract tests          -> producer/consumer compatibility
end-to-end tests        -> a few critical deployed journeys
load/security tests     -> nonfunctional limits and abuse behavior
production telemetry    -> actual environment validation
```

The build is a function from reviewed source plus pinned inputs to an artifact and evidence. Hidden machine state, changing dependency metadata, undeclared tools, timezone, locale, or network downloads make that function unstable.

> **Specification boundary:** The Java compiler and `--release` option define language/API targeting behavior for supported releases, but Maven, Gradle, JUnit, analysis plugins, coverage tools, and dependency resolution are separate tools. Their defaults and configuration models are version-sensitive.

Core terminology

- **Test oracle:** Rule that decides whether observed behavior is correct.
- **Fixture:** Controlled state and data used by a test.
- **Unit test:** Test of a small behavior with in-process collaborators controlled.
- **Integration test:** Test of interaction with a real boundary or multiple configured components.
- **Contract test:** Test that a provider and consumer agree on a boundary schema/behavior.
- **Test double:** Replacement collaborator, including fake, stub, spy, or mock.
- **Fake:** Working simplified implementation, such as an in-memory repository.
- **Stub:** Supplies predetermined responses.
- **Mock:** Verifies configured interactions.
- **Flaky test:** Test whose result varies without a relevant product change.
- **Hermetic build/test:** Runs from declared inputs without relying on uncontrolled external state.
- **Toolchain:** Selected JDK used to compile, test, or run build tasks.
- **Transitive dependency:** Dependency brought in by another dependency.
- **Lock/checksum:** Mechanism constraining resolved versions or artifact integrity.
- **SBOM:** Software bill of materials describing shipped components.

Detailed mechanics

Test behavior, not implementation shape

Assert public outcomes, emitted records, committed state, and meaningful collaborator contracts. Avoid asserting private method calls, exact incidental SQL order, or every getter invocation. Such tests prevent refactoring without protecting users.

Use Arrange-Act-Assert or Given-When-Then to make the causal boundary visible. One test can assert several aspects of one outcome, but avoid multiple unrelated acts. Name the scenario and expected behavior: `expired_token_is_rejected` communicates more than `testValidate2`.

Cover happy paths, boundaries, invalid input, duplicate/retry behavior, and failure recovery. Use equivalence partitions rather than enumerating random examples. Parameterized tests express a behavior table. Property-based tests generate broad input and shrink failures, but properties need meaningful invariants; "does not throw" is often weak.

Determinism

Inject `Clock` rather than calling the current time throughout domain code. Supply a seeded or fake random source when exact identity matters. Use temporary directories supplied by the test framework. Set explicit charset, locale, timezone, and ordering when they affect output.

Do not use arbitrary sleeps to coordinate concurrent tests. Use latches, barriers, futures with bounded waits, controllable schedulers, or event probes. A test that passes 1,000 times does not prove a data race absent. Use concurrency stress/model-checking tools where appropriate and retain a simple invariant test.

Isolate mutable state. Static caches, singleton clients, thread locals, environment variables, and database rows can leak between tests. Parallel test execution magnifies hidden sharing. Every test should either own state or use namespaced fixtures and cleanup that survives failure.

Test doubles and boundaries

Use a fake when stateful behavior matters, a stub for a simple response, and a mock when the interaction itself is the contract. Mocking a class you own can reveal that its API is awkward; mocking a vendor SDK throughout the domain spreads vendor details.

Do not mock the database to prove SQL, transaction, constraint, type, or isolation behavior. Run integration tests against the same database engine and compatible version. An in-memory substitute may implement different SQL and concurrency semantics. Containerized test dependencies improve realism but add image, startup, platform, and supply-chain concerns.

Consumer-driven contract tests can catch incompatible request/response changes before deployment, but they do not prove availability or semantic correctness. Maintain a small end-to-end path through authentication, network, persistence, and messaging for the most critical journeys.

Exception and async tests

Assert the exception type, safe message or error code, causal information, and state after failure. A test that only expects "some exception" accepts too much. For futures and reactive/asynchronous work, await with a bounded deadline and assert both result and cancellation/failure propagation.

Test retry logic with a fake sleeper/scheduler rather than wall-clock delay. Assert maximum attempts, backoff sequence, idempotency identity, and non-retryable failure. Test that resources close after exceptions using a small instrumented fake or integration metric.

Coverage, mutation, and static evidence

Line or branch coverage shows executed structure, not assertion quality. A high percentage can coexist with no meaningful oracle. Use coverage to find untested risk, not as a universal quality score.

Mutation testing changes operations and asks whether tests fail. Surviving meaningful mutations expose weak assertions, but equivalent mutations and runtime cost require triage. Apply it to critical pure logic rather than every generated accessor.

Static analysis complements tests by exploring paths without running inputs. Useful categories include nullness, resource leaks, ignored return values, concurrency annotations, insecure APIs, bug patterns, API compatibility, style, and architecture dependency rules. Enable compiler lint checks and treat new serious findings as failures. Suppress narrowly with a reason and scope; a global exclusion turns evidence off.

> **Tooling note:** SpotBugs, Error Prone, NullAway, Checkstyle, PMD, JaCoCo, mutation tools, and architecture-test libraries have distinct models and false positives. Pin versions and understand whether a rule is sound, heuristic, source-level, or bytecode-level before making it a gate.

Build reproducibility and Java targeting

Use the Maven or Gradle wrapper so developers and CI invoke a reviewed tool version. Select JDK toolchains explicitly. Compiling on JDK 21 with source compatibility 17 alone can accidentally call Java 21 APIs. Use the compiler's `--release 17` mechanism, through the build tool, when the artifact must run on Java 17.

Pin plugin versions. Avoid dynamic dependency versions and mutable artifact repositories. Verify checksums/signatures according to organizational policy. Reproducible archives also require stable timestamps, file order, metadata, and generated content. Byte-for-byte reproducibility is stronger than "the same source compiled successfully."

Separate fast checks from slower integration, contract, and performance stages while keeping one release provenance chain. Generated sources, annotation processors, and code generators are build dependencies with executable power. Pin and review them like runtime libraries.

Dependency resolution and governance

Declare direct dependencies explicitly rather than relying on a transitive one by accident. Use scopes/configurations correctly so test tools do not ship in runtime artifacts and compile-only APIs are present where needed. Inspect the resolved graph, not only the build file.

Maven and Gradle resolve version conflicts differently and behavior changes with plugins/configuration. A BOM or platform aligns a tested family but does not prove compatibility with every application dependency. Locking improves repeatability; it also means update automation must deliberately refresh and validate locks.

For each shipped component consider:

- origin and artifact integrity;
- supported version and release cadence;
- known vulnerabilities and exploitability in the application;
- license and distribution obligations;
- transitive footprint and duplicate functionality;
- Java baseline, native code, reflection, and framework compatibility; and
- maintenance or replacement plan.

A vulnerability scanner match is a triage input. Confirm the actual artifact, affected version, reachable feature, deployment exposure, compensating control, and vendor fix. Do not ignore a critical issue because no public exploit is known, and do not upgrade blindly without compatibility tests.

CI pipeline design

A practical pipeline runs formatting/checks, compilation, unit tests, static analysis, packaging, integration/contract tests, dependency and secret scans, artifact signing/attestation, and selected deployment tests. Order cheap high-signal failures early. Run from a clean checkout with least-privilege credentials.

Cache immutable artifacts using keys that include relevant locks and tool versions. A cache must accelerate resolution, not alter it. Publish one promoted artifact rather than rebuilding independently for each environment.

TRACE IT | Worked Java example

This invitation policy injects time, making expiry tests deterministic:

JAVA (continued 1/2)

```java
import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.junit.jupiter.api.Assertions.assertTrue;

import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.time.ZoneOffset;
import org.junit.jupiter.api.Test;

record Invitation(Instant expiresAt) {
    boolean isValidAt(Instant instant) {
        return instant.isBefore(expiresAt);
    }
}

final class InvitationService {
    private final Clock clock;
    private final Duration lifetime;

    InvitationService(Clock clock, Duration lifetime) {
        this.clock = java.util.Objects.requireNonNull(clock);
        if (lifetime == null || lifetime.isZero() || lifetime.isNegative()) {
            throw new IllegalArgumentException("lifetime must be positive");
        }
        this.lifetime = lifetime;
```

JAVA (continued 2/2)

```java
    }

    Invitation create() {
        return new Invitation(clock.instant().plus(lifetime));
    }
}

final class InvitationServiceTest {
    private static final Instant NOW = Instant.parse("2026-01-01T12:00:00Z");
    private static final Clock CLOCK = Clock.fixed(NOW, ZoneOffset.UTC);

    @Test
    void invitation_is_valid_before_expiry_but_not_at_expiry() {
        Invitation invitation = new InvitationService(CLOCK, Duration.ofMinutes(15))
                .create();

        assertTrue(invitation.isValidAt(NOW.plusSeconds(899)));
        assertFalse(invitation.isValidAt(NOW.plusSeconds(900)));
    }

    @Test
    void non_positive_lifetime_is_rejected() {
        assertThrows(IllegalArgumentException.class,
                () -> new InvitationService(CLOCK, Duration.ZERO));
    }
}
```

The snippet uses JUnit Jupiter APIs and needs a pinned JUnit dependency in the test configuration. It tests boundary semantics at exactly the expiry instant without sleeping.

TRACE IT | Execution or memory walkthrough

The fixed clock always returns noon. `create` adds 15 minutes, producing an immutable invitation expiring at 12:15. At 12:14:59 the test instant is strictly before expiry, so validity is true. At exactly 12:15 the strict comparison is false. The test documents a half-open validity interval rather than leaving `<=` versus `<` implicit.

The second test never constructs invalid service state. It checks the public constructor boundary and exact exception type. No mock verifies that `clock.instant()` was called once; call count is an implementation detail unless the contract requires one time snapshot.

Memory and threads are local to each test. The static fixed clock and instant are immutable. If tests changed JVM default timezone instead, parallel tests could interfere. Explicit UTC data avoids that global-state dependency.

Complexity and performance

Test feedback time is roughly:

TEXT

```
sum(test execution + fixture setup + dependency startup)
divided by safe parallelism, plus scheduling and build overhead
```

Unit tests should normally be milliseconds or less, while real database startup and migration dominate integration stages. Reuse can reduce cost but risks state leakage. Prefer suite-level infrastructure with per-test transaction/schema/namespace isolation when behavior remains realistic.

Build complexity grows with modules, annotation processing, generated code, dependency graph, and cache misses. Parallelism helps independent tasks until CPU, memory, disk, ports, database connections, or test isolation becomes limiting.

Evidence	Strength	Blind spot
unit tests	precise logic and edge cases	real wiring and infrastructure
integration tests	driver/framework/database behavior	full deployment/network path
contract tests	boundary compatibility	provider correctness/availability
end-to-end	critical real journey	slow diagnosis and combinatorial coverage
static analysis	broad path/pattern scan	runtime/environment behavior
coverage	executed code map	oracle quality
mutation	assertion sensitivity	cost/equivalent changes

WATCH OUT | Edge cases and common mistakes

- Asserting private methods or exact call sequences instead of behavior.
- Mocking the database and claiming transaction or SQL correctness.
- Using sleeps, current time, default timezone, unordered collections, or unseeded randomness.
- Sharing ports, rows, files, static caches, or environment state across parallel tests.
- Catching an exception in a test but never failing when no exception occurs.
- Treating coverage percentage as correctness.
- Quarantining flaky tests indefinitely without ownership and deadline.
- Running every test as an end-to-end test and producing slow, ambiguous failures.
- Compiling with a new JDK's APIs while claiming an older runtime target.
- Omitting plugin and annotation-processor versions.
- Depending accidentally on a transitive library.
- Trusting a BOM, lock file, or vulnerability scanner as proof of safety.
- Rebuilding different artifacts for staging and production.
- Placing production credentials or personal data in fixtures, logs, or test reports.

Production engineering notes

Give flaky tests the urgency of production defects. Capture seed, schedule, environment, and artifacts; assign an owner; fix root cause or temporarily quarantine with a deadline and visible risk. Blind retries hide nondeterminism and inflate CI cost.

Keep tests close to risk. Database migrations need forward/backward and realistic-volume tests. Serialization needs golden compatibility fixtures. Idempotent message handling needs duplicate and crash-window tests. Security controls need negative and abuse cases. Performance-sensitive changes need controlled benchmarks, not fixed nanosecond assertions on shared CI.

Use dependency update automation to open small reviewed changes with release notes, resolved graph diff, tests, and rollback plan. Generate an SBOM from the shipped artifact, not only declared dependencies. Scan container base images and build plugins as well as Java libraries.

Protect the build system: least-privilege tokens, isolated untrusted pull-request jobs, approved repositories, checksum verification, no secret echo, and signed/promoted artifacts. A compromised build dependency executes with CI authority.

TRY IT | Interview questions and model answers

What should you unit test versus integration test?

Unit-test domain decisions and boundaries with controlled collaborators. Integration-test behavior owned by the database, driver, serializer, framework configuration, or network protocol. Use a few end-to-end tests for critical deployed journeys.

When is a mock appropriate?

When interaction with a collaborator is itself the contract or a difficult boundary must return a controlled result. Prefer state/output assertions and fakes when possible; avoid mocking implementation details.

How do you make time-dependent tests reliable?

Inject `Clock` or a domain time source, use fixed instants and explicit zones, and test just before, at, and after boundaries without sleeping.

What does `--release 17` do when compiling on JDK 21?

It selects the supported Java 17 language/API target model so code cannot accidentally link against newer standard APIs, while generating the corresponding class-file target. It does not validate third-party runtime compatibility.

How do you manage dependency vulnerabilities?

Inventory the shipped graph, verify the affected artifact/version and reachable feature, assess exposure and controls, upgrade or mitigate promptly, run compatibility tests, and document exceptions with expiration.

Why are reproducible builds important?

They make artifacts traceable to reviewed inputs, support verification and incident response, and prevent hidden machine state or mutable dependencies from changing what ships.

TRY IT | Exercises

1. Refactor a test using `Thread.sleep(1_000)` into one using a fake scheduler or synchronization primitive.
2. Design tests for a money transfer: pure invariant tests, real database transaction tests, duplicate-command tests, and one end-to-end path.
3. Write a property for a sorting function that is stronger than "output has the same size."
4. Configure a Java 21 build toolchain that produces a Java 17-compatible library and explain third-party dependency checks still needed.
5. Inspect a hypothetical resolved graph with two logging API versions and propose an alignment and verification plan.
6. Create a CI stage order that gives fast feedback while protecting release provenance and secrets.

RECAP | Chapter summary

Testing is a layered evidence system. Deterministic unit tests protect domain behavior, integration tests validate real boundaries, contract tests protect schemas, and a few end-to-end tests validate critical journeys. Build wrappers, toolchains, release targeting, pinned plugins, locks, checksums, and promoted artifacts make delivery reproducible. Static analysis and coverage guide risk but do not replace oracles. Dependency governance must inspect the shipped transitive graph, security, licensing, origin, and compatibility.

TRY IT | Revision checklist

- [ ] I select test scope from the behavior and failure risk.
- [ ] I assert public outcomes rather than private implementation steps.
- [ ] I control clock, randomness, locale, timezone, files, ports, and concurrency.
- [ ] I know when to use a fake, stub, mock, real database, or contract test.
- [ ] I interpret coverage, mutation, and static findings as evidence, not proof.
- [ ] I use wrappers, pinned plugins, explicit toolchains, and `--release` targeting.
- [ ] I inspect and lock the resolved dependency graph and generate an artifact SBOM.
- [ ] I treat flaky tests and build-system security as production reliability concerns.

SDE-2 CORE CHAPTER

Chapter 4 - Secure and Reliable Java

Learning objectives

By the end of this chapter, you should be able to:

- threat-model Java service assets, actors, trust boundaries, and abuse cases;
- validate input and prevent injection, unsafe deserialization, path, XML, SSRF, and resource-exhaustion defects;
- apply authentication, authorization, secrets, cryptography, and logging at appropriate boundaries;
- design deadlines, bounded retries, idempotency, bulkheads, backpressure, and graceful degradation;
- connect security and reliability through least privilege, bounded work, safe failure, and recovery; and
- present a practical secure-development and incident-readiness checklist for Java 17 and Java 21 services.

WHY IT WORKS | Why this matters at SDE-2

Security and reliability fail at assumptions: a field was "internal," a timeout meant rollback, a queue would stay small, a URL was trusted after parsing, or a dependency was safe because it was popular. Attackers deliberately exercise edge cases that normal traffic rarely reaches, and outages turn ordinary retries and logs into amplifiers.

SDE-2 engineers should build controls into API and system design rather than bolt them onto controllers. They must understand that validation is context-specific, authorization belongs on every protected action, cryptography requires lifecycle and key management, and graceful degradation cannot violate money, privacy, or integrity rules.

WHY IT WORKS | First-principles model

Threat modeling begins with a data-flow view:

TEXT

```
external actor -> network boundary -> parser -> application policy
               -> database/cache -> message -> downstream service
```

At each crossing ask:

1. What assets and invariants matter?
2. Who can supply or observe data?

3 How can identity, confidentiality, integrity, availability, or auditability fail?

4 Which prevention, detection, response, and recovery controls apply?

5 What residual risk and operational signal remain?

Reliability uses a similar model. Work consumes finite CPU, memory, threads, connections, file descriptors, queue slots, and downstream capacity. Every admission path needs a bound and every remote effect needs an outcome model.

TEXT

```
safe service = validated authority + bounded work + explicit deadlines
             + idempotent recovery + observable state + tested rollback
```

> **Specification boundary:** Java supplies type safety, memory safety for ordinary managed code, cryptographic and networking APIs, and runtime checks. It does not make application authorization, parsers, native code, reflection, dependencies, secrets, SQL, files, or distributed protocols automatically safe. Provider algorithms and TLS/JCA defaults vary by JDK/vendor/version.

Core terminology

- **Asset:** Data, capability, availability, money, identity, or reputation requiring protection.
- **Trust boundary:** Point where data or authority crosses between different trust assumptions.
- **Authentication:** Establishing a principal's identity.
- **Authorization:** Deciding whether that principal may perform one action on one resource.
- **Least privilege:** Granting only capabilities required for a limited purpose and duration.
- **Defense in depth:** Independent controls that limit failure of one layer.
- **Injection:** Untrusted data interpreted as commands or structure in another language.
- **SSRF:** Server-side request forgery that makes a service access unintended network resources.
- **Deserialization:** Converting external representation into in-memory values or objects.
- **Deadline:** Absolute latest completion time propagated across calls.
- **Backpressure:** Slowing, rejecting, or shedding producers when capacity is exhausted.
- **Bulkhead:** Resource partition that limits one workload's blast radius.
- **Circuit breaker:** State machine that temporarily rejects calls after evidence of dependency failure.

Detailed mechanics

Validate by context and bound work

Decode using an explicit charset and media type, enforce total bytes and nesting before expensive allocation, then validate type, range, length, format, and cross-field domain rules. Prefer allow-lists and typed parsers. A string safe as display text is not automatically safe in SQL, HTML, shell, LDAP, a file path, a regular expression, or a log.

Parameterize SQL values. Avoid executing operating-system commands; when unavoidable, pass an argument list to a fixed executable rather than building a shell string, validate every argument, set a working directory/environment deliberately, and impose timeout/output limits.

Regular expressions can consume superlinear CPU, so bound input and choose safe patterns. Compressed uploads need entry, expanded-byte, nesting, path, and ratio limits. `Content-Length` is not a sufficient bound.

Paths, URLs, XML, and serialization

Normalize paths to reject simple traversal, but remember symlinks and validation-use races. Store user content under generated names and least-privilege roots. Archive extraction validates every final destination and refuses links or special files unless explicitly supported.

For outbound URLs, strictly allow schemes, hosts, ports, and paths; account for private/link-local/loopback addresses, redirects, and DNS rebinding. Network egress controls provide an independent layer. Blocking one textual hostname is not SSRF prevention.

Disable XML external entities and external DTD/schema access unless narrowly required. Defaults vary, so test the actual parser with malicious fixtures.

Never use native Java deserialization for untrusted data. Isolate unavoidable legacy use with allow-list and graph limits. Explicit schemas still need polymorphism, size, duplicate-field, and version policies.

Authentication and authorization

Authenticate at a verified boundary, then authorize each operation and resource. Resolve tenant and ownership from trusted server-side state, not solely from request fields.

Use short-lived, narrowly scoped credentials and validate signature, issuer, audience, time, and key rotation through the protocol library. Enforce application-level authorization on user, administrative, and background paths, not only in a UI or gateway.

Secrets and cryptography

Keep secrets out of source, build logs, command lines, heap dumps, URLs, and ordinary metrics. Obtain them from an approved secret system, grant least privilege, rotate, audit access, and design clients to refresh without full outages. Environment variables can be exposed by process diagnostics and are not a universal secure store.

Use maintained protocols and approved JCA/JCE libraries. Do not invent encryption, nonce, password-hashing, signature, or certificate rules. Passwords need a salted, work-factor KDF selected by

current policy. Random tokens need `SecureRandom`; UUID uniqueness does not automatically provide secret unpredictability.

Authenticated encryption requires unique nonces per key and verified tags before plaintext use. Keys need rotation, revocation, and access separation. Never install a "trust all" TLS manager or disable hostname verification in production.

> **Vendor boundary:** Available JCA providers, algorithm names, key-store support, TLS versions/cipher defaults, certificate revocation, and hardware acceleration differ by distribution and configuration. Pin policy and test handshakes on every supported runtime.

Logging, errors, and audit

Log events and identifiers needed for diagnosis, not raw request bodies, access tokens, passwords, card data, session cookies, encryption keys, or broad object `toString` output. Sanitize control characters to prevent log forging. Bound message length and metric-label cardinality.

Return stable public errors with minimal detail and preserve restricted causal context internally. Keep access-controlled audit events separate from debug logs when required.

Deadlines and bounded retry

Propagate an absolute deadline so each layer sees remaining time. Configure connection acquisition, connect, read/write, database statement, and queue waits within it. Cancellation must release resources. A timeout is an unknown remote outcome, not proof of rollback.

Wall clocks can jump and hosts can disagree. Propagate an absolute deadline for cross-process policy, account for clock skew, and use a monotonic elapsed-time source for local budget enforcement when precision matters.

Retry only classified transient failures and idempotent operations, or reuse a stable idempotency key. Bound attempts and elapsed time, use exponential backoff with jitter, and honor server retry hints only within policy. Retrying at multiple stack layers multiplies attempts. Centralize ownership of retries and measure attempts versus logical requests.

Circuit breakers reduce repeated calls to a failing dependency but add a state machine and can reject recovery probes. Bulkheads bound concurrency per dependency or tenant. Rate limits and admission control reject before expensive work. Bounded queues force a documented response: block within a deadline, reject, spill durably, shed lower priority, or degrade safely.

Idempotency and state transitions

An idempotency key identifies one logical command, is scoped to principal/operation, and stores a request fingerprint plus completed or in-progress outcome. Reuse with a different payload must be rejected. Expiry must exceed plausible client retry windows or the old effect can be repeated later.

Database unique constraints or compare-and-set updates enforce concurrency. Consumers record message identity in the same transaction as their business effect. Exactly-once transport wording does not make an email, payment, or arbitrary database side effect exactly once.

Safe degradation and recovery

Degrade only features whose loss does not violate authorization, integrity, billing, or durability. Recommendations can disappear; authorization normally fails closed.

Liveness should answer whether the process should be restarted; readiness should answer whether it should receive new traffic. Making liveness depend on every downstream can cause restart storms. Exact probe semantics belong to the orchestrator and service design.

On shutdown, stop admission, mark unready, allow bounded in-flight completion, checkpoint or return leased work, close clients/executors, and exit before the platform's hard deadline. Test forced termination and duplicate processing.

Backups are not recovery until restores are tested. Define recovery objectives, key recovery, schema repair, incident authority, evidence preservation, and credential rotation.

Java-specific attack surface

Reflection, agents, annotation processors, JNI, native libraries, and dynamic loading expand capability. Minimize them and treat processors as executable dependencies. The Security Manager is deprecated for removal in Java 17/21; isolate hostile code at process/container and OS boundaries.

TRACE IT | Worked Java example

This retrier makes limits, classification, deadline, sleeping, and jitter explicit:

JAVA (continued 1/2)

```java
import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.util.concurrent.TimeoutException;
import java.util.function.LongSupplier;

record RetryPolicy(int maxAttempts, Duration initialBackoff,
        Duration maxBackoff) {
    public RetryPolicy {
        if (maxAttempts < 1 || initialBackoff == null || maxBackoff == null
                || initialBackoff.isNegative() || initialBackoff.isZero()
                || maxBackoff.compareTo(initialBackoff) < 0) {
            throw new IllegalArgumentException("invalid retry policy");
        }
    }
}

@FunctionalInterface interface CheckedOperation<T> {
    T run(Instant deadline) throws Exception;
}

@FunctionalInterface interface RetryClassifier {
    boolean isRetryable(Exception failure);
```

JAVA (continued 2/2)

```
}

@FunctionalInterface interface Sleeper {
    void sleep(Duration duration) throws InterruptedException;
}

@FunctionalInterface interface Jitter {
    Duration apply(Duration upperBound);
}

final class BoundedRetrier {
    private final Clock clock;
    private final LongSupplier nanoTime;
    private final Sleeper sleeper;
    private final Jitter jitter;

    BoundedRetrier(Clock clock, LongSupplier nanoTime,
            Sleeper sleeper, Jitter jitter) {
        this.clock = java.util.Objects.requireNonNull(clock);
        this.nanoTime = java.util.Objects.requireNonNull(nanoTime);
        this.sleeper = java.util.Objects.requireNonNull(sleeper);
        this.jitter = java.util.Objects.requireNonNull(jitter);
    }
```

The bounded retry loop continues inside the same `BoundedRetrier` class:

JAVA (continued 1/2)

```
    <T> T execute(Instant deadline, RetryPolicy policy,
            RetryClassifier classifier, CheckedOperation<T> operation)
            throws Exception {
        Duration backoff = policy.initialBackoff();
        Duration initialBudget = Duration.between(clock.instant(), deadline);
        long startedAtNanos = nanoTime.getAsLong();
        Exception lastFailure = null;

        for (int attempt = 1; attempt <= policy.maxAttempts(); attempt++) {
            Duration remaining = remainingBudget(
                    deadline, initialBudget, startedAtNanos);
            if (remaining.isNegative() || remaining.isZero()) {
                TimeoutException timeout = new TimeoutException("deadline exceeded");
                if (lastFailure != null) timeout.addSuppressed(lastFailure);
                throw timeout;
            }
            try {
                return operation.run(deadline);
            } catch (InterruptedException interrupted) {
                Thread.currentThread().interrupt();
                throw interrupted;
            } catch (Exception failure) {
                lastFailure = failure;
                if (!classifier.isRetryable(failure)
                        || attempt == policy.maxAttempts()) {
```

JAVA (continued 2/2)

```
                    throw failure;
                }
            }

            remaining = remainingBudget(deadline, initialBudget, startedAtNanos);
            Duration cap = min(backoff, remaining);
            if (cap.isNegative() || cap.isZero()) {
                TimeoutException timeout = new TimeoutException("no retry budget");
                timeout.addSuppressed(lastFailure);
                throw timeout;
            }
            Duration delay = jitter.apply(cap);
            if (delay == null || delay.isNegative() || delay.compareTo(cap) > 0) {
                throw new IllegalStateException("jitter violated its contract");
            }
            try {
                sleeper.sleep(delay);
            } catch (InterruptedException interrupted) {
                Thread.currentThread().interrupt();
                throw interrupted;
            }
            backoff = doubleCapped(backoff, policy.maxBackoff());
        }
        throw new AssertionError("unreachable");
    }
```

The duration helpers complete `BoundedRetrier`:

JAVA

```
    private Duration remainingBudget(Instant deadline,
            Duration initialBudget, long startedAtNanos) {
        Duration wallRemaining = Duration.between(clock.instant(), deadline);
        long elapsedNanos = nanoTime.getAsLong() - startedAtNanos;
        if (elapsedNanos < 0) {
            return Duration.ZERO;
        }
        Duration monotonicRemaining = initialBudget.minusNanos(elapsedNanos);
        return min(wallRemaining, monotonicRemaining);
    }

    private static Duration min(Duration left, Duration right) {
        return left.compareTo(right) <= 0 ? left : right;
    }

    private static Duration doubleCapped(Duration value, Duration maximum) {
        return value.compareTo(maximum.dividedBy(2)) > 0
                ? maximum : value.multipliedBy(2);
    }
}
```

A production constructor can inject `System::nanoTime` as the monotonic ticker; tests inject a deterministic ticker. The wrapper stops when either the propagated wall-clock deadline or the original monotonic elapsed-time budget is exhausted, so a backward wall-clock adjustment cannot extend retries. A production jitter implementation might choose a random duration between zero and the cap; tests inject exact delays. The operation receives the absolute deadline and must apply its own per-call timeouts. The wrapper cannot interrupt an arbitrary blocking API merely because either clock reaches the deadline.

TRACE IT | Execution or memory walkthrough

Assume `maxAttempts = 3`, initial backoff 100 ms, maximum 1 second, and five seconds remain. The first transient failure is classified retryable. Jitter returns 60 ms, so the sleeper waits 60 ms. The next cap is 200 ms. If the second call succeeds, execution returns after two attempts.

If failure is an authentication rejection, the classifier returns false and no retry occurs. If the thread is interrupted during the operation or sleep, the retrier restores the interrupt flag and propagates interruption rather than treating cancellation as transient failure.

If only 40 ms remains after a failure under either clock, the backoff cap becomes 40 ms; jitter cannot exceed it. The next operation still receives the same absolute deadline. A client with a 500 ms socket timeout would violate that deadline, so the operation must derive its timeout from remaining time.

The retrier holds only one last exception, policy values, and local durations: `O(1)` state. The remote side still needs an idempotency key because the first timed-out attempt may have committed.

Complexity and performance

With at most `a` attempts, local control work is `O(a)` and state is `O(1)`. Total worst-case delay is bounded by the deadline and attempt limit, not merely by the backoff sum. Without limits, retries can grow traffic by a multiplier at every layer:

```text
3 gateway attempts x 3 service attempts x 3 client attempts = 27 calls
```

Security bounds turn attacker-controlled dimensions into configured limits:

Dimension	Unbounded risk	Control
request bytes/nesting	heap/CPU exhaustion	parser and transport limits
regex/input length	CPU exhaustion	safe pattern plus length bound
queue/concurrency	memory and downstream collapse	admission and bulkhead
decompression	disk/heap explosion	expanded-byte and ratio limits
retries	outage amplification	classifier, jitter, attempts, deadline
metric labels/logs	memory/storage/cardinality	allow-list and truncation

Cryptographic cost is intentional. Password KDF parameters consume CPU/memory to resist guessing and must be capacity-tested. Do not lower them reactively without a security decision; protect login capacity with rate limiting and dedicated resource budgets.

WATCH OUT | Edge cases and common mistakes

- Validating a string once and reusing it in a different output context.
- Trusting client-supplied tenant/resource IDs after authentication without authorization.
- Concatenating SQL, shell, path, URL, LDAP, or log structure from untrusted data.
- Allowing URL redirects or DNS resolution to bypass an SSRF host check.
- Enabling XML external entities or broad polymorphic deserialization.
- Treating encryption without authentication as integrity protection.
- Reusing an AEAD nonce or storing keys beside ciphertext with identical access.
- Logging tokens, passwords, request bodies, secrets, or diagnostic dumps without controls.
- Enforcing elapsed retry budgets only with an adjustable wall clock and ignoring clock skew.
- Retrying permanent failures, interruption, or non-idempotent commands.
- Implementing retries at several layers without a total budget.
- Using unbounded queues or cached thread pools to absorb overload.
- Failing open when authorization, billing, or integrity dependencies fail.
- Making liveness depend on a downstream service and causing restart storms.
- Running untrusted code under the deprecated Security Manager as a sandbox.
- Assuming a successful backup job proves restoration.

Production engineering notes

Maintain threat models and owners for controls, alerts, keys, exceptions, and recovery. Review changes involving authorization, parsing, cryptography, file/network access, native code, and serialization.

Default to denial, bounded work, safe parsers, verified TLS, and no production debug surface. Make overrides narrow, audited, and temporary.

Measure logical requests, attempts, rejection, queue age, circuit state, timeout phase, authorization decisions, and dependency latency without leaking secrets or high-cardinality IDs.

Practice restores, outages, key/certificate rotation, duplicates, overload, and shutdown. Runbooks cover containment, evidence, credentials, communication, and rollback.

TRY IT | Interview questions and model answers

How do you secure a new endpoint?

Define assets and actors, authenticate the principal, authorize the action on the resource, bound and validate parsing, parameterize downstream queries, set deadline and capacity, redact observability, make side effects idempotent, and test abuse and failure paths.

What is the difference between authentication and authorization?

Authentication establishes who the caller is. Authorization decides whether that caller can perform this action on this resource under current policy. A valid identity can still be unauthorized.

Why are retries dangerous?

They amplify load, increase latency, and can duplicate side effects. Retry only transient failures within one bounded owner and deadline, with jitter and idempotency or reconciliation.

How would you prevent SSRF?

Use strict URL parsing and allow-lists, validate scheme/host/port, control DNS/private addresses and redirects, enforce egress network policy, bound response/time, and avoid accepting arbitrary URLs when an identifier can be used.

Should a service fail open or closed?

It depends on the invariant. Authorization, integrity, and payment controls normally fail closed. Optional presentation features may degrade. Make the decision explicit and test dependency failure.

What is defense in depth?

Independent controls limit one another's failure: application URL policy plus network egress restrictions, prepared SQL plus least-privilege database credentials, and parser limits plus request admission.

How do you handle an unknown outcome after timeout?

Reuse a stable idempotency key and query status or retry the same logical operation safely. Do not create a new command identity and assume the first attempt rolled back.

TRY IT | Exercises

1. Threat-model a file-upload endpoint: include parser, archive, path, malware, authorization, quotas, storage, and download behavior.
2. Design idempotency storage for a payment command, including payload fingerprint, in-progress state, response replay, and expiry.
3. Calculate worst-case calls when three layers each retry twice after the original attempt. Redesign with one retry owner.
4. Extend the retrier test design with a fake clock, sleeper, jitter, transient failure, deadline exhaustion, and interruption.
5. Review an outbound webhook feature for SSRF, secret signing, timeouts, retries, redirect policy, and tenant isolation.
6. Write graceful-shutdown steps for a message consumer that leases work and may receive duplicate delivery.

RECAP | Chapter summary

Secure and reliable Java systems make trust and capacity explicit. Validate data for its destination context, authorize every protected resource action, use approved cryptography and secret lifecycle, and restrict dangerous parsing and dynamic capabilities. Bound bytes, time, concurrency, queues, retries, and cardinality. Treat timeouts as unknown outcomes and use idempotency for recovery. Layer preventive controls with telemetry, graceful failure, least privilege, tested restore, and incident practice.

TRY IT | Revision checklist

- [ ] I can identify assets, actors, trust boundaries, abuse cases, and residual risk.
- [ ] I validate input by context and bound bytes, depth, CPU, concurrency, and output.
- [ ] I can prevent SQL/shell/path/XML/deserialization and SSRF classes of defects.
- [ ] I separate authentication from resource-level authorization.
- [ ] I use approved secret, password, encryption, nonce, key, and TLS practices.
- [ ] I propagate deadlines and bound one retry owner with classification, jitter, and idempotency.
- [ ] I design bulkheads, backpressure, safe degradation, and graceful shutdown.
- [ ] I protect logs, metrics, diagnostics, dependencies, builds, backups, and incident evidence.

Part II - Practice and Handoff

Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: refactor cohesive boundaries
- Interview Core: design a service and test strategy
- SDE-2 Follow-up: transactions, retries, and observability
- Challenge: threat-model and harden a production workflow

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous volume: 18E - Advanced Java E: Performance, Diagnostics, and GC Incidents

Series index

Next volume: 18G - Advanced Java G: Question Bank, Study Plan, and Reference

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Complete the learning steps in order when rebuilding from fundamentals, or enter at the first step whose readiness checks expose a gap. Stable volume numbers remain in filenames; the steps below show the recommended reading order. Click a title when your PDF viewer permits local sibling-file links.

STEP	FOCUSED LEARNING PATH
1	Java Foundations for Problem Solving
2	Time and Space Complexity
3	Number Systems and Math Foundations for DSA Interviews
4	Bit Manipulation in Java
5	Loop Mastery, Patterns, and Index Calculations
6	Arrays and Array Problem-Solving Patterns
7	Strings and String Problem-Solving Patterns
8	Hashing: Maps, Sets, Frequency, and Prefix State
9	Recursion and Backtracking in Java
10	Linked Lists: Pointer Reasoning and Mutation
11	Stacks, Queues, Deques, and Monotonic Patterns
12	Binary Search: Bounds, Answers, and Invariants
13	Trees, Binary Search Trees, and Tries
14	Heaps, Priority Queues, Selection, and Top-K
15	Graphs: Traversal, Ordering, Paths, and Union-Find
16	Greedy Algorithms: Recognition, Proofs, and Scheduling
17	Dynamic Programming: State, Transitions, and Optimization
18	Advanced Java Engineering and the SDE-2 Interview (Parts A-J)

Learning Step 3 uses a linked Number Systems foundations volume and workbook; the final advanced stage uses a ten-part collection, including a three-volume backend specialist track. These splits keep every file focused and printable.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Volume 18 of 18 - Part F of J. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.